Lecture-22: Streams & File I/O

You need to read a **number** into your program.

Easy... but **where does it come from?**

Where Does the Data Come From?

- The **keyboard** (the user types it)

Where Does the Data Come From?

- The **keyboard** (the user types it)
- A **file** on disk

Where Does the Data Come From?

- The **keyboard** (the user types it)
- A **file** on disk
- A **string** already in memory

Where Does the Data Come From?

- The **keyboard** (the user types it)
- A **file** on disk
- A **string** already in memory
- A **network socket**

Where Does the Data Come From?

- The **keyboard** (the user types it)
- A **file** on disk
- A **string** already in memory
- A **network socket**

Each source has a **completely different** low-level API.

And in *every* case the bytes are just **characters**: `"3.14"` that must be converted to a `double`.

The Problem & The Solution

Problem:

- A separate code path for every source?
- Manual character-to-type conversion?

Not practical.

The Problem & The Solution

Problem:

- A separate code path for every source?
- Manual character-to-type conversion?

Not practical.

Solution (Bjarne Stroustrup):

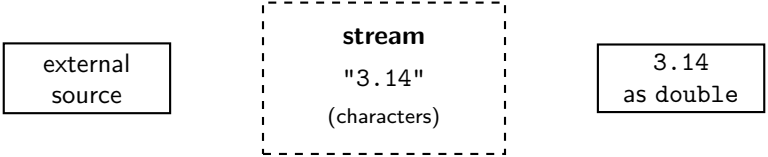
> *"Designing a general* **input/output** *facility for a programming language is notoriously difficult. . . "*
>
> — so he built **streams**.

One interface: >> to read, << to write.

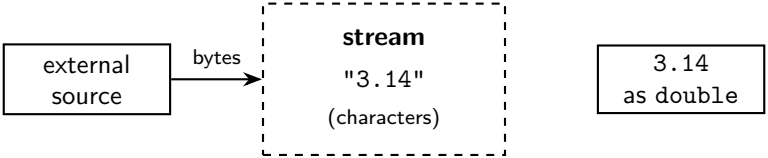
A Stream is an Abstraction

A **stream** = an object that represents
a **sequence of characters** flowing into or out of your program.

A Stream is an Abstraction (cont.)

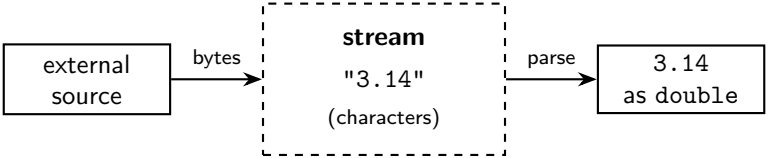


A Stream is an Abstraction (cont.)



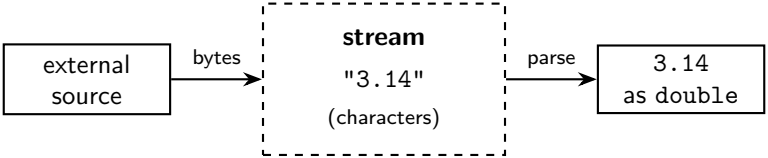
- **Left:** can be *anything* — keyboard, file, string, socket.

A Stream is an Abstraction (cont.)



- **Left:** can be *anything* — keyboard, file, string, socket.
- **Middle:** the stream — a buffer of characters.
- **Right:** your program gets a **typed** value.

A Stream is an Abstraction (cont.)



- **Left:** can be *anything* — keyboard, file, string, socket.
- **Middle:** the stream — a buffer of characters.
- **Right:** your program gets a **typed** value.

Key: You write code against the stream interface once; the backend can change without changing your code.

A Familiar Stream: `std::cout`

You already use a stream:

```
std::cout << "Hello, World" << std::endl;
```

A Familiar Stream: `std::cout`

```
std::cout << "Hello, World" << std::endl;
```

- `std::cout` is an **instance** of `std::ostream`
  — the standard **output** stream.

A Familiar Stream: `std::cout`

```
std::cout << "Hello, World" << std::endl;
```

- `std::cout` is an **instance** of `std::ostream`
  — the standard **output** stream.
- `<<` is the **insertion operator**:
  "push these characters *into* the stream."

A Familiar Stream: `std::cout`

```
std::cout << "Hello, World" << std::endl;
```

- `std::cout` is an **instance** of `std::ostream`
  — the standard **output** stream.
- `<<` is the **insertion operator**:
  "push these characters *into* the stream."
- The stream then forwards them to the **console**.

A Familiar Stream: `std::cout`

```
std::cout << "Hello, World" << std::endl;
```

- `std::cout` is an **instance** of `std::ostream` — the standard **output** stream.
- `<<` is the **insertion operator**: "push these characters *into* the stream."
- The stream then forwards them to the **console**.
- You never call a low-level "write to terminal" function — the stream **hides** it from you.

The Mirror Image: `std::cin`

`std::cin` is an instance of `std::istream` — the standard **input** stream.

```
int main() {
    double pi;
    std::cin >> pi;                    // read from console
    std::cout << pi / 2 << '\n';    // write to console
}
```

The Mirror Image: `std::cin`

`std::cin` is an instance of `std::istream` — the standard **input** stream.

```
int main() {
    double pi;
    std::cin >> pi;                    // read from console
    std::cout << pi / 2 << '\n';    // write to console
}
```

- >> is the **extraction operator**:
  "pull characters *out of* the stream and parse them."
- The stream reads `"3.14"` (characters) and gives you the `double` `3.14`.

Two Operators, Two Directions

Operator	**Name**	**Data flows...**
<<	insertion	*into* the stream (writing)
>>	extraction	*out of* the stream (reading)

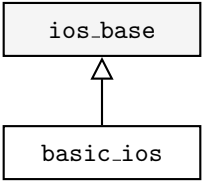
Two Operators, Two Directions

Operator	Name	Data flows...
<<	insertion	*into* the stream (writing)
>>	extraction	*out of* the stream (reading)

Mnemonic: the arrows point in the direction the data flows.

- `std::cout << x;` data goes → into the stream.
- `std::cin >> x;` data comes ← out of the stream into `x`.

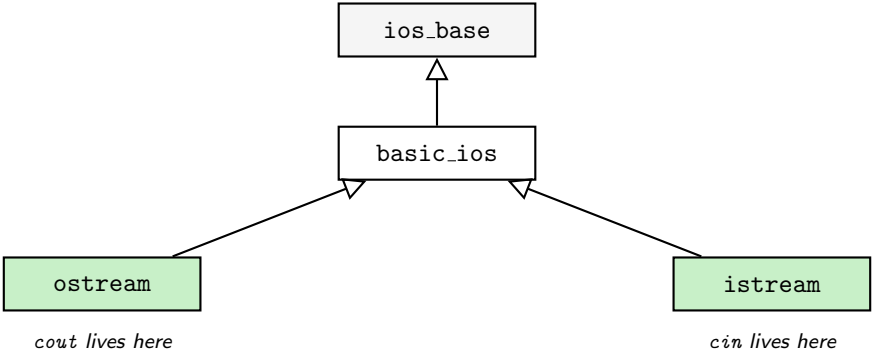
The Hierarchy — Step 1: The Base



- `ios_base` holds format flags, error state bits, locale.
- `basic_ios` adds the stream buffer.

Everything inherits from these.

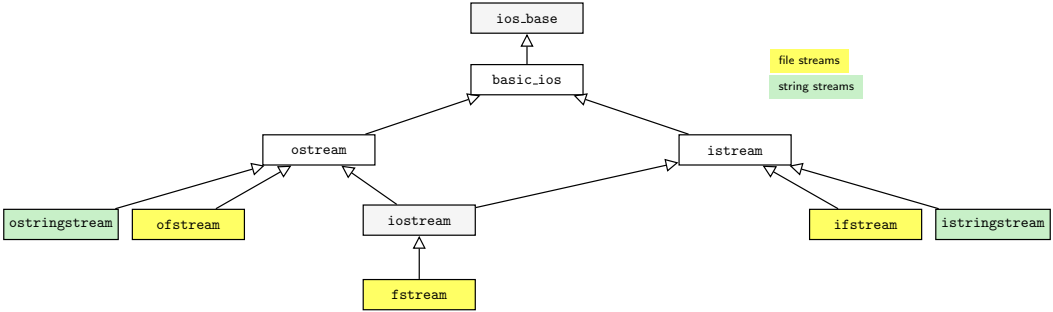
The Hierarchy — Step 2: Input & Output



`ostream` — output (writing)
operator <<

`istream` — input (reading)
operator >>

The Hierarchy — Step 3: Concrete Streams



Same << and >> operators work everywhere.
That is the entire point of the inheritance.

Key Insight: Streams are Polymorphic

Because `ifstream` inherits from `istream`, **anything you can do with `cin`, you can do with a file stream.**

From the keyboard

```
int x;
std::cin >> x;
std::string line;
std::getline(std::cin, line);
```

Key Insight: Streams are Polymorphic

Because `ifstream` inherits from `istream`, **anything you can do with `cin`, you can do with a file stream.**

From the keyboard

```
int x;
std::cin >> x;
std::string line;
std::getline(std::cin, line);
```

From a file — same code!

```
int x;
std::ifstream fin("data.txt");
fin >> x;
std::string line;
std::getline(fin, line);
```

Key Insight: Streams are Polymorphic

Because `ifstream` inherits from `istream`, **anything you can do with `cin`, you can do with a file stream.**

From the keyboard

```
int x;
std::cin >> x;
std::string line;
std::getline(std::cin, line);
```

From a file — same code!

```
int x;
std::ifstream fin("data.txt");
fin >> x;
std::string line;
std::getline(fin, line);
```

Design principle: Write functions that accept `istream&` or `ostream&` — they work with *any* stream backend.

```
void process(std::istream& in) {   // cin, ifstream, istringstream
    std::string word;
    while (in >> word) { /* ... */ }
}
```

`std::stringstream`: Strings as Streams

What? A stream whose backend is a `std::string` in memory.

`std::stringstream`: Strings as Streams

What? A stream whose backend is a `std::string` in memory.

Why?

- **Parsing** — split a string into typed pieces with >>.

`std::stringstream`: Strings as Streams

What? A stream whose backend is a `std::string` in memory.

Why?

- **Parsing** — split a string into typed pieces with >>.
- **Building** — assemble a formatted string with <<.

`std::stringstream`: Strings as Streams

What? A stream whose backend is a `std::string` in memory.

Why?

- **Parsing** — split a string into typed pieces with >>.
- **Building** — assemble a formatted string with <<.
- **Type conversion** — `"42"` $\leftrightarrow$ 42 with no manual work.

`std::stringstream`: Strings as Streams

What? A stream whose backend is a `std::string` in memory.

Why?

- **Parsing** — split a string into typed pieces with >>.
- **Building** — assemble a formatted string with <<.
- **Type conversion** — `"42"` $\leftrightarrow$ 42 with no manual work.

Key: `stringstream` inherits from `iostream`, so it supports *both* << and >>.

Walkthrough: Parsing with `istringstream`

Goal: Split `"Bjarne Stroustrup C++ fun"` into separate variables.

Step 1 — Create the string we want to parse:

```
std::string quote = "Bjarne Stroustrup C++ fun";
```

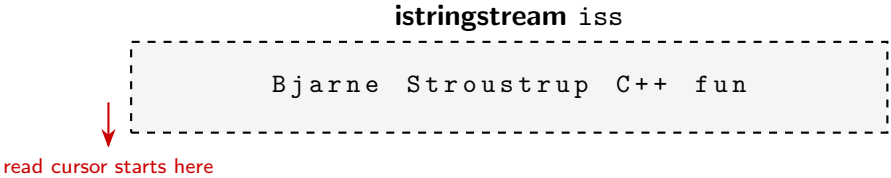
```
Bjarne Stroustrup C++ fun
```

`quote` is just a flat string — no structure yet.

Walkthrough: Parsing with `istringstream` (cont.)

Step 2 — Wrap the string in an `istringstream`:

```
std::string quote = "Bjarne Stroustrup C++ fun";
std::istringstream iss(quote);    // treat it as an input stream
```



Now we can use >> on `iss` — exactly like `cin`.

Walkthrough: Parsing with `istringstream` (cont.)

Step 3 — Extract tokens with >>:

```
std::string first, last, language, extra;
iss >> first;     // "Bjarne"
```

Walkthrough: Parsing with `istringstream` (cont.)

Step 3 — Extract tokens with >>:

```
std::string first, last, language, extra;
iss >> first;     // "Bjarne"
```

```
iss >> last;      // "Stroustrup"
```

Walkthrough: Parsing with `istringstream` (cont.)

Step 3 — Extract tokens with >>:

```
std::string first, last, language, extra;
iss >> first;     // "Bjarne"
```

```
iss >> last;      // "Stroustrup"
```

```
iss >> language;  // "C++"
```

Walkthrough: Parsing with `istringstream` (cont.)

Step 3 — Extract tokens with >>:

```
std::string first, last, language, extra;
iss >> first;     // "Bjarne"
```

```
iss >> last;      // "Stroustrup"
```

```
iss >> language;  // "C++"
```

```
iss >> extra;     // "fun"
```

Walkthrough: Parsing with `istringstream` (cont.)

Step 3 — Extract tokens with >>:

```
std::string first, last, language, extra;
iss >> first;     // "Bjarne"
```

```
iss >> last;      // "Stroustrup"
```

```
iss >> language;  // "C++"
```

```
iss >> extra;     // "fun"
```

>> skips whitespace and extracts the next token. **No manual parsing.**

```
std::cout << first << " " << last
          << " said " << language
          << " is " << extra << "!\n";
// Output: Bjarne Stroustrup said C++ is fun!
```

Walkthrough: Building with `ostringstream`

Goal: Format mixed types into one string.

Walkthrough: Building with `ostringstream`

Goal: Format mixed types into one string.

```
std::string name = "Alice";
int    age   = 21;
double gpa   = 8.75;
```

Walkthrough: Building with `ostringstream`

Goal: Format mixed types into one string.

```
std::string name = "Alice";
int    age   = 21;
double gpa   = 8.75;
```

```
std::ostringstream oss;              // empty output string-stream
oss << name << " (" << age << ") has GPA " << gpa;
```

Walkthrough: Building with `ostringstream`

Goal: Format mixed types into one string.

```
std::string name = "Alice";
int    age   = 21;
double gpa   = 8.75;
```

```
std::ostringstream oss;              // empty output string-stream
oss << name << " (" << age << ") has GPA " << gpa;
```

```
std::string report = oss.str();      // pull the built string out
std::cout << report << '\n';
// Output: Alice (21) has GPA 8.75
```

Walkthrough: Building with `ostringstream`

Goal: Format mixed types into one string.

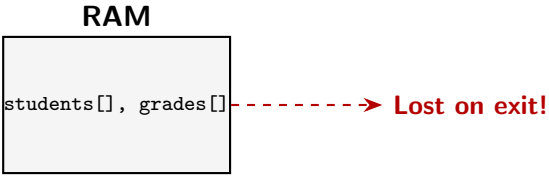
```
std::string name = "Alice";
int    age   = 21;
double gpa   = 8.75;
```

```
std::ostringstream oss;              // empty output string-stream
oss << name << " (" << age << ") has GPA " << gpa;
```

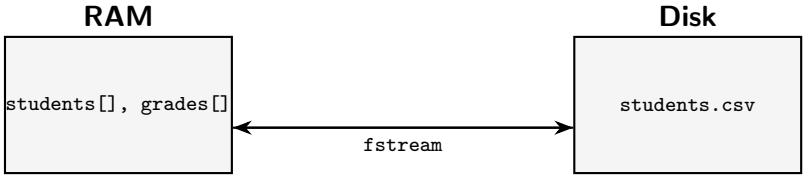
```
std::string report = oss.str();      // pull the built string out
std::cout << report << '\n';
// Output: Alice (21) has GPA 8.75
```

The same << you use for `std::cout` — but the "console" is now an **in-memory string**.

Why File I/O?



Why File I/O?



Streams bridge the gap — use `ofstream` to write, `ifstream` to read.
The interface is the **same** << **and** >> you already know.

Writing to a File: `ofstream` — Step 1

Open the file and check for errors:

```
#include <fstream>
#include <iostream>

int main() {
    std::ofstream fout("report.txt");   // creates or truncates
    if (!fout) {                        // ALWAYS check
        std::cerr << "Cannot open report.txt\n";
        return 1;
    }
```

- The constructor opens the file.
- If opening fails, the stream is in a **fail state** — the `if(!fout)` check catches it.

Writing to a File: `ofstream` — Step 2

Write to it exactly like `std::cout`:

```
    fout << "Student Grade Report\n";
    fout << "====================\n";
    fout << "Alice   : " << 92 << "\n";
    fout << "Bob     : " << 78 << "\n";
}  // fout closes automatically here (RAII)
```

Writing to a File: `ofstream` — Step 2

Write to it exactly like `std::cout`:

```
    fout << "Student Grade Report\n";
    fout << "====================\n";
    fout << "Alice   : " << 92 << "\n";
    fout << "Bob     : " << 78 << "\n";
}  // fout closes automatically here (RAII)
```

Result on disk (`report.txt`):

```
Student Grade Report
====================
Alice   : 92
Bob     : 78
```

Writing to a File: `ofstream` — Step 2

Write to it exactly like `std::cout`:

```
    fout << "Student Grade Report\n";
    fout << "====================\n";
    fout << "Alice   : " << 92 << "\n";
    fout << "Bob     : " << 78 << "\n";
}  // fout closes automatically here (RAII)
```

Result on disk (`report.txt`):

```
Student Grade Report
====================
Alice   : 92
Bob     : 78
```

RAII: The destructor of `ofstream` calls `close()` automatically.
No need for an explicit `fout.close()` (unless you want to check write errors early).

File Open Modes

Flag	Name	Behaviour
`ios::in`	Input	Open for reading
`ios::out`	Output	Open for writing (default for `ofstream`)
`ios::app`	Append	Seek to end before each write
`ios::trunc`	Truncate	Discard existing content
`ios::binary`	Binary	No newline translation

File Open Modes

Flag	Name	Behaviour
`ios::in`	Input	Open for reading
`ios::out`	Output	Open for writing (default for `ofstream`)
`ios::app`	Append	Seek to end before each write
`ios::trunc`	Truncate	Discard existing content
`ios::binary`	Binary	No newline translation

Appending to a log file:

```
std::ofstream log("server.log", std::ios::app);
log << "[2026-04-12] User logged in\n";
// Previous content preserved; new line appended
```

File Open Modes

Flag	Name	Behaviour
`ios::in`	Input	Open for reading
`ios::out`	Output	Open for writing (default for `ofstream`)
`ios::app`	Append	Seek to end before each write
`ios::trunc`	Truncate	Discard existing content
`ios::binary`	Binary	No newline translation

Appending to a log file:

```
std::ofstream log("server.log", std::ios::app);
log << "[2026-04-12] User logged in\n";
// Previous content preserved; new line appended
```

Combining modes: `std::fstream f("data.bin", ios::in | ios::out | ios::binary);`

Reading from a File: `ifstream` — Word by Word

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream fin("poem.txt");
    if (!fin) { std::cerr << "Error\n"; return 1; }

    std::string word;
    int count = 0;
    while (fin >> word) {    // extraction as loop condition
        ++count;
    }
    std::cout << "Word count: " << count << "\n";
}
```

Reading from a File: `ifstream` — Word by Word

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream fin("poem.txt");
    if (!fin) { std::cerr << "Error\n"; return 1; }

    std::string word;
    int count = 0;
    while (fin >> word) {    // extraction as loop condition
        ++count;
    }
    std::cout << "Word count: " << count << "\n";
}
```

Note: `fin >> word` returns a reference to the stream. In a boolean context it converts to `true` if the last operation **succeeded**.

Reading Line by Line with `getline`

```
std::ifstream fin("students.csv");
if (!fin) { std::cerr << "Error\n"; return 1; }

std::string header;
std::getline(fin, header);   // skip the header row

std::string line;
while (std::getline(fin, line)) {
    std::cout << "Row: " << line << "\n";
}
```

Reading Line by Line with `getline`

```
std::ifstream fin("students.csv");
if (!fin) { std::cerr << "Error\n"; return 1; }

std::string header;
std::getline(fin, header);   // skip the header row

std::string line;
while (std::getline(fin, line)) {
    std::cout << "Row: " << line << "\n";
}
```

Why `getline` instead of >>?

- >> skips whitespace and stops at the *first* whitespace.
- `getline` reads *everything* up to the delimiter (default `'\n'`).

Walkthrough: Parsing a CSV — The File

Suppose `students.csv` contains:

```
name,age,gpa
Alice,21,3.8
Bob,22,3.5
Carol,20,3.9
```

Goal: Read each row into a `Student` struct.

Walkthrough: Parsing a CSV — The File

Suppose `students.csv` contains:

```
name,age,gpa
Alice,21,3.8
Bob,22,3.5
Carol,20,3.9
```

Goal: Read each row into a `Student` struct.

```
struct Student {
    std::string name;
    int age;
    double gpa;
};
```

Walkthrough: Parsing a CSV — Outer Loop

Step 1: Open the file, skip the header, read one line at a time.

```
std::ifstream fin("students.csv");
std::string line;
std::getline(fin, line);   // skip "name,age,gpa"

while (std::getline(fin, line)) {
    // line = "Alice,21,3.8" on first iteration
    // ... parse this line ...
}
```

We cannot use >> here because >> splits on whitespace, not commas.

Walkthrough: Parsing a CSV — Inner Parse

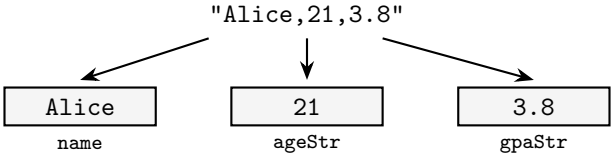
Step 2: Wrap the line in an `istringstream` and use `getline` with `','`.

```
while (std::getline(fin, line)) {
    std::istringstream iss(line);       // "Alice,21,3.8"

    std::string name, ageStr, gpaStr;
    std::getline(iss, name,   ',');     // "Alice"
    std::getline(iss, ageStr, ',');     // "21"
    std::getline(iss, gpaStr, ',');     // "3.8"
```

Walkthrough: Parsing a CSV — Inner Parse

Step 2: Wrap the line in an `istringstream` and use `getline` with `','`.

```
while (std::getline(fin, line)) {
    std::istringstream iss(line);       // "Alice,21,3.8"

    std::string name, ageStr, gpaStr;
    std::getline(iss, name,   ',');     // "Alice"
    std::getline(iss, ageStr, ',');     // "21"
    std::getline(iss, gpaStr, ',');     // "3.8"
```



Walkthrough: Parsing a CSV — Convert Types

Step 3: Convert strings to numeric types and store the result.

```
    int age    = std::stoi(ageStr);    // "21" -> 21
    double gpa = std::stod(gpaStr);   // "3.8" -> 3.8
    std::cout << name << " age=" << age
              << " gpa=" << gpa << "\n";
}
```

Walkthrough: Parsing a CSV — Convert Types

Step 3: Convert strings to numeric types and store the result.

```
    int age    = std::stoi(ageStr);    // "21" -> 21
    double gpa = std::stod(gpaStr);   // "3.8" -> 3.8
    std::cout << name << " age=" << age
              << " gpa=" << gpa << "\n";
}
```

Conversion functions (from `<string>`):

`std::stoi(s)`	string → `int`
`std::stod(s)`	string → `double`
`std::stof(s)`	string → `float`
`std::to_string(n)`	numeric → `string`

Parsing CSV into struct (C++)

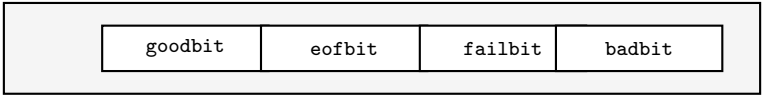
```cpp
#include <fstream>
#include <sstream>
#include <string>

struct Student {
std::string name;
int age;
double gpa;
};

int main() {
std::ifstream fin("students.csv");
if (!fin) return 1;
std::string line;
std::getline(fin, line); // skip header
```

```cpp
while (std::getline(fin, line)) {
std::istringstream iss(line);
    std::string name, ageStr, gpaStr;

    std::getline(iss, name, ',');
    std::getline(iss, ageStr, ',');
    std::getline(iss, gpaStr, ',');

    Student s;
    s.name = name;
    s.age = std::stoi(ageStr);
    s.gpa = std::stod(gpaStr);

    std::cout << s.name
              << " age=" << s.age
              << " gpa=" << s.gpa << "\n";
}
}
```

Stream State Flags

Every stream carries internal state bits:

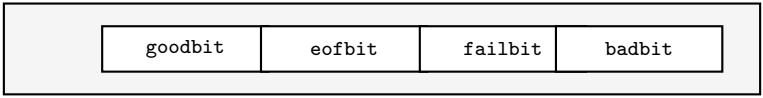
Internal state of every stream



Stream State Flags

Every stream carries internal state bits:

Internal state of every stream

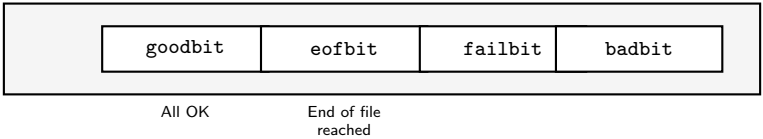


All OK

Stream State Flags

Every stream carries internal state bits:

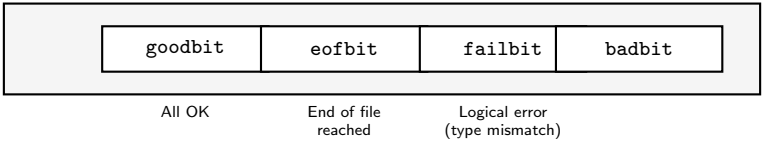
Internal state of every stream



Stream State Flags

Every stream carries internal state bits:

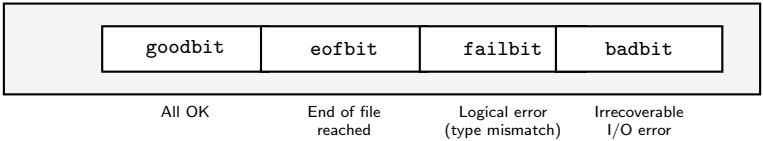
Internal state of every stream



Stream State Flags

Every stream carries internal state bits:

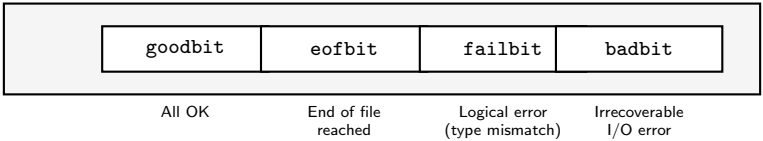
Internal state of every stream



Stream State Flags

Every stream carries internal state bits:

Internal state of every stream



Method	True when...
`.good()`	No error bits set
`.eof()`	Hit end of file
`.fail()`	`failbit` *or* `badbit` set
`.clear()`	Resets all error flags

The Classic EOF Bug

BUG — Do NOT do this

```
std::ifstream fin("nums.txt");
int x;
while (!fin.eof()) {  // BAD!
    fin >> x;
    std::cout << x << "\n";
}
```

The Classic EOF Bug

BUG — Do NOT do this

```
std::ifstream fin("nums.txt");
int x;
while (!fin.eof()) {  // BAD!
    fin >> x;
    std::cout << x << "\n";
}
```

If file contains `10 20 30` ⟨*space*⟩
the last value prints **twice**.

Why? After reading 30
`eof()` is still `false` —
it turns `true` only after the
next failed read.

The Classic EOF Bug

BUG — Do NOT do this

```
std::ifstream fin("nums.txt");
int x;
while (!fin.eof()) {  // BAD!
    fin >> x;
    std::cout << x << "\n";
}
```

If file contains `10 20 30` ⟨*space*⟩ the last value prints **twice**.

Why? After reading 30 `eof()` is still `false` — it turns `true` only after the *next* failed read.

CORRECT patterns

```
// Pattern A
while (fin >> x) {
    std::cout << x << "\n";
}

// Pattern B
std::string line;
while (std::getline(fin, line)) {
    std::cout << line << "\n";
}
```

Always make the **read operation** itself the loop condition.

Pitfall: Mixing >> and getline

BUG

```
// input.txt:
// 42
// Hello World

std::ifstream fin("input.txt");
int n;
fin >> n;
std::string line;
std::getline(fin, line);
std::cout << "n=" << n
          << " line=[" << line << "]\n";
```

Pitfall: Mixing >> and getline

BUG

```
// input.txt:
// 42
// Hello World

std::ifstream fin("input.txt");
int n;
fin >> n;
std::string line;
std::getline(fin, line);
std::cout << "n=" << n
          << " line=[" << line << "]\n";
```

Output:

```
n=42 line=[]
```

Pitfall: Mixing >> and getline

BUG

```
// input.txt:
// 42
// Hello World

std::ifstream fin("input.txt");
int n;
fin >> n;
std::string line;
std::getline(fin, line);
std::cout << "n=" << n
          << " line=[" << line << "]\n";
```

Output:

```
n=42 line=[]
```

Why? `fin >> n` reads 42 but leaves `'\n'` in the buffer. `getline` reads that leftover newline — empty!

Pitfall: Mixing >> and `getline`

BUG

```
// input.txt:
// 42
// Hello World

std::ifstream fin("input.txt");
int n;
fin >> n;
std::string line;
std::getline(fin, line);
std::cout << "n=" << n
          << " line=[" << line << "]\n";
```

Output:

```
n=42 line=[]
```

Why? `fin >> n` reads 42 but leaves `'\n'` in the buffer. `getline` reads that leftover newline — empty!

Fix:

```
fin >> n;
fin.ignore(
  std::numeric_limits<
    std::streamsize>::max(),
  '\n');
std::getline(fin, line);
```

Discards all remaining characters in the input buffer up to and including the next newline.

Predict the Output

Suppose `data.txt` contains:

```
42 hello
3.14
```

```
std::ifstream fin("data.txt");
int n;
std::string s;
double d;
fin >> n >> s >> d;
std::cout << n << "|"
          << s << "|"
          << d << "\n";
```

Predict the Output

Suppose `data.txt` contains:

```
42 hello
3.14
```

```
std::ifstream fin("data.txt");
int n;
std::string s;
double d;
fin >> n >> s >> d;
std::cout << n << "|"
          << s << "|"
          << d << "\n";
```

Output:

```
42|hello|3.14
```

Key: >> skips whitespace *including newlines* between extractions.

Predict the Output #2

```
std::string data = "10:20:30";
std::istringstream iss(data);
std::string token;
while (std::getline(iss, token, ':')) {
    std::cout << "[" << token << "] ";
}
std::cout << "\n";
```

Predict the Output #2

```
std::string data = "10:20:30";
std::istringstream iss(data);
std::string token;
while (std::getline(iss, token, ':')) {
    std::cout << "[" << token << "] ";
}
std::cout << "\n";
```

Output:

```
[10] [20] [30]
```

`getline(iss, token, ':')` uses `':'` as the delimiter instead of `'\n'`.

Worked Example: Sum Numbers from a Line

Problem: The user types a line like `10 20 30 40`.
Compute the sum — but the count of numbers is **unknown**.

Worked Example: Sum Numbers from a Line

Problem: The user types a line like `10 20 30 40`.
Compute the sum — but the count of numbers is **unknown**.

We cannot write `std::cin >> a >> b >> c >> d;` because we do not know how many values there are.

Worked Example: Sum Numbers from a Line

Problem: The user types a line like `10 20 30 40`.
Compute the sum — but the count of numbers is **unknown**.

We cannot write `std::cin >> a >> b >> c >> d;` because we do not know how many values there are.

Strategy:

1. Read the *entire line* with `getline`.
2. Wrap it in an `istringstream`.
3. Loop with >> until the stream is exhausted.

Worked Example — Step 1: Read the Line

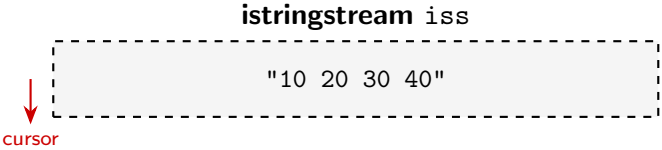
```
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::string line;
    std::getline(std::cin, line);   // e.g. "10 20 30 40"
```

```
line = "10 20 30 40"
```

The whole line is now in one string.

Worked Example — Step 2: Wrap in `istringstream`

```
std::istringstream iss(line);
```



Now we can use `iss >> value` in a loop — exactly like reading from `cin`.

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {     // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	value	sum	Remaining in stream
1	10	10	"20 30 40"

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	`value`	`sum`	**Remaining in stream**
1	10	10	"20 30 40"
2	20	30	"30 40"

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	`value`	`sum`	**Remaining in stream**
1	10	10	`"20 30 40"`
2	20	30	`"30 40"`
3	30	60	`"40"`

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	`value`	`sum`	**Remaining in stream**
1	10	10	"20 30 40"
2	20	30	"30 40"
3	30	60	"40"
4	40	100	"" (empty)

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	`value`	`sum`	**Remaining in stream**
1	10	10	"20 30 40"
2	20	30	"30 40"
3	30	60	"40"
4	40	100	"" (empty)
5	—	100	extraction **fails** → loop ends

Worked Example — Step 3: Extract & Sum

```
    int value, sum = 0;
    while (iss >> value) {      // stops when extraction fails
        sum += value;
    }
    std::cout << "sum = " << sum << '\n';
}
```

Trace:

Iteration	`value`	`sum`	**Remaining in stream**
1	10	10	"20 30 40"
2	20	30	"30 40"
3	30	60	"40"
4	40	100	"" (empty)
5	—	100	extraction **fails** → loop ends

Output: `sum = 100`

Why This Example Matters

- `std::cin` is an `istream`. `std::istringstream` is *also* an `istream`.

Why This Example Matters

- `std::cin` is an `istream`. `std::istringstream` is *also* an `istream`.
- The **same loop** (`while (stream >> value)`) works for both.

Why This Example Matters

- `std::cin` is an `istream`. `std::istringstream` is *also* an `istream`.
- The **same loop** (`while (stream >> value)`) works for both.
- The loop terminates automatically when the stream is **exhausted** (the stream converts to `false`).

Why This Example Matters

- `std::cin` is an `istream`. `std::istringstream` is *also* an `istream`.
- The **same loop** (`while (stream >> value)`) works for both.
- The loop terminates automatically when the stream is **exhausted** (the stream converts to `false`).
- We wrote the parsing code **once**; it works whether the numbers came from the console, a file, or a string.

Why This Example Matters

- `std::cin` is an `istream`. `std::istringstream` is *also* an `istream`.
- The **same loop** (`while (stream >> value)`) works for both.
- The loop terminates automatically when the stream is **exhausted** (the stream converts to `false`).
- We wrote the parsing code **once**; it works whether the numbers came from the console, a file, or a string.

The abstraction paid off.

One interface. Many backends. That is the power of streams.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.
3. `cout`/`cin` are just instances of stream classes;
   `ofstream`, `ifstream`, `stringstream` use the **same operators**.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.
3. `cout`/`cin` are just instances of stream classes;
   `ofstream`, `ifstream`, `stringstream` use the **same operators**.
4. `stringstream` lets you reuse stream code on a plain string —
   great for parsing and formatting.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.
3. `cout`/`cin` are just instances of stream classes;
   `ofstream`, `ifstream`, `stringstream` use the **same operators**.
4. `stringstream` lets you reuse stream code on a plain string —
   great for parsing and formatting.
5. For file I/O: `ofstream` to write, `ifstream` to read.
   Always check `if(!stream)`.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.
3. `cout`/`cin` are just instances of stream classes;
   `ofstream`, `ifstream`, `stringstream` use the **same operators**.
4. `stringstream` lets you reuse stream code on a plain string —
   great for parsing and formatting.
5. For file I/O: `ofstream` to write, `ifstream` to read.
   Always check `if(!stream)`.
6. Use `getline` + `istringstream` + `getline(..., ',')` to parse CSV data.

Key Takeaways

1. A **stream** is a uniform interface to a sequence of characters.
2. << writes *into* a stream; >> reads *out of* a stream.
3. `cout`/`cin` are just instances of stream classes;
   `ofstream`, `ifstream`, `stringstream` use the **same operators**.
4. `stringstream` lets you reuse stream code on a plain string —
   great for parsing and formatting.
5. For file I/O: `ofstream` to write, `ifstream` to read.
   Always check `if(!stream)`.
6. Use `getline` + `istringstream` + `getline(..., ',')` to parse CSV data.
7. **Never** use `while(!fin.eof())` — use the read as the loop condition.

Summary: When to Use What

Task	**Tool**	**Key pattern**
Console output	`cout`	`cout << data`
Console input	`cin`	`cin >> x` or `getline(cin,line)`
Write to file	`ofstream`	`fout << data`
Read from file	`ifstream`	`while(fin >> x)`
Parse a string	`istringstream`	`iss >> x` or `getline(iss,tok,',')`
Build a string	`ostringstream`	`oss << x; oss.str()`
Generic I/O function	`istream&`, `ostream&`	Accepts *any* input/output stream

References

1. B. Stroustrup, *The C++ Programming Language*, 4th ed., Chapter 38 (I/O Streams).

2. Stanford CS106L Course Reader, Chapter 3: *Streams* Slide